FADENA

LICENCIATURA EN CIBERDEFENSA

DISPOSITIVOS REMOTOS E NTERNET DE LAS COSAS

ACTIVIDAD OBLIGATORIA I

AÑO: 2026

ACTIVIDAD OBLIGATORIA:
SIMULACIONES HTTP Y MQTT + CAPTURA CON WIRESHARK

Objetivo de la actividad

El propósito es:

1. Forquear y clonar el repositorio a tu PC.
2. Ejecutar simulaciones de comunicación IoT usando diferentes protocolos (HTTP y MQTT).
3. Capturar el tráfico de red con Wireshark.
4. Analizar las trazas capturadas, identificando:
    - Si los mensajes (payloads) se ven en texto plano o codificados.
    - Qué ocurre en cada capa de red:
        - Capa 7: aplicación (HTTP, MQTT),
        - Capa 4: transporte (TCP/UDP, puertos),
        - Capa 3: red (IP origen/destino),
        - Capa 2: enlace (MAC),
    - Direcciones, puertos, diferencias entre protocolos.

---

1 Fork del repositorio en tu cuenta de GitHub

1. Abre el enlace:
   ☞ Repositorio iot_repo en GitHub
2. Inicia sesión en GitHub (o crea una cuenta si no tienes).
3. Haz clic en “Fork” (esquina superior derecha) para copiar el repo a tu cuenta.
4. Espera a que se complete el fork — tendrás una copia personal del proyecto.

---

2 Clonar el repositorio en tu PC

Una vez hecho el fork:

1. En tu repositorio fork, haz clic en Code → copia la URL (HTTPS).
2. En tu terminal, elige una carpeta y ejecuta:

```
git clone https://github.com/TU_USUARIO/iot_repo.git
cd iot_repo
```

Reemplaza TU_USUARIO por tu nombre de usuario de GitHub.

---

3 Instalar dependencias y preparar Python

Este proyecto usa Python 3.12+ y el gestor uv o pip (según README).
En tu terminal:

```
# Si usas pip
pip install -r requirements.txt

# O instalar uv (si se recomienda)
pip install uv
```

---

4 Ejecutar simulaciones de protocolos

El repositorio contiene varios scripts para simular IoT con distintos protocolos.
Según el README:

□ HTTP

En una terminal:

```
uv run uvicorn lectura:app --port 8000 --reload
```

En otra terminal:

```
uv run python sensor.py
```

Esto simula un servidor que recibe datos y un sensor que los envía.

□ MQTT (público o local)

4.1 Ejecutar suscriptor MQTT

```
uv run python lectura_mq.py
```

4.2 Ejecutar publicador MQTT

```
uv run python sensor_mq.py
```

Estos scripts demuestran el protocolo MQTT (Message Queuing Telemetry Transport), muy usado en IoT por su eficiencia y modelo publicación/suscripción.

📌 Opcional: hay scripts con simulación de errores (lectura_er y sensor_er).

5️⃣ Capturar con Wireshark

Abre Wireshark y selecciona la interfaz de red que usarás:

- Para tráfico local (localhost): usa la interfaz Loopback (lo) o any.
- Inicia la captura antes de ejecutar los scripts.

🔍 Filtros recomendados

Protocolo	Filtro en Wireshark
HTTP	tcp.port == 8000
MQTT	mqtt o tcp.port == 1883
MQTT seguro (TLS)	tcp.port == 8883
WebSocket	websocket
CoAP (UDP)	coap o udp.port == 5683

Estos filtros te permiten ver solo paquetes relevantes durante la simulación.

6️⃣ Análisis de la captura en Wireshark

Ahora que ya tienes tráfico capturado, analiza cada nivel:

Capa 7 – Aplicación

- HTTP

- Verás solicitudes y respuestas HTTP en texto claro.
- Mensajes típicos:
    - GET/POST, cabeceras HTTP, cuerpo con datos de sensor en texto/JSON.
- Puedes ver directamente la URL, método y contenido si no está cifrado.

- MQTT

- MQTT está sobre TCP.
- Dependiendo de si usas un broker local o público, puede ser:
    - Texto plano: paquetes MQTT visibles (CONNECT, PUBLISH, SUBSCRIBE).
    - Codificado: si usas TLS en 8883, el contenido estará encriptado (no podrás leerlo sin la clave).

Observa:

- Topic al que se publica.
- Payload (en MQTT en texto plano solo si no hay cifrado).
- Si es MQTT seguro (TLS), Wireshark mostrará el protocolo pero no el contenido.

Capa 4 – Transporte

- HTTP usa TCP (puerto 8000 por defecto en este repo).
- MQTT usa TCP:
    - Puerto 1883 (no cifrado) — mensajes legibles.
    - Puerto 8883 (TLS) — contenido cifrado.
- Observa:
    - Direcciones IP de origen/destino.
    - Puertos de sesión.
    - Flags TCP (SYN, ACK, etc.).

Capa 3 – Red

- Paquetes IP:
    - IP origen → IP destino.
    - Puedes ver TTL, tamaño y tipo de paquete.

- Identifica:
    - Si se trata de tráfico interno (localhost 127.0.0.1) o externo.
    - Latencia, reenvíos o retransmisiones TCP.

Capa 2 – Enlace

- Identifica direcciones MAC (si no es tráfico loopback).
- En red local, puede servir para saber qué dispositivo envió qué.
- Si estás capturando en loopback, no habrá direcciones MAC físicas.

7️⃣ Verificando si el mensaje se ve en texto plano

✓ HTTP

- Es texto claro: cabeceras y cuerpo pueden verse sin decodificar.

✓ MQTT en puerto 1883

- Es texto legible de comandos y payload MQTT.

✓ MQTT en puerto 8883 (TLS)

- El contenido está cifrado y Wireshark *no mostrará texto plano* sin claves privadas.

📌 Conclusión esperada

En tu informe deberías mencionar:

✅ Qué protocolo capturaste (HTTP vs MQTT).
✅ Si el contenido está en claro o cifrado.
✅ Qué viste en cada capa (Aplicación/Transporte/Red/Enlace).
✅ Direcciones, puertos y tipo de mensaje.
✅ ¿Los mensajes contienen datos legibles? (sí en HTTP y MQTT sin TLS; no si usas TLS).